

Relatório — Gates de conteúdo sine qua non (verificável, real, aplicável, replicável, mensurável, escalável, cientificamente embasado)

Data: 09-08-2026 Escopo: Fábrica Agêntica de Publicações — critérios de validação de CONTEÚDO dos materiais gerados (independente do tipo de obra).

1. Diagnóstico — os conteúdos seguem hoje estes critérios?

Resposta: NÃO. O que existe hoje é validação de estrutura, auto-consistência interna e formato de entrega. Os critérios de mérito de conteúdo não são regras travas.

Critério	Status	Evidência
Verificáveis	Parcial — verifica-se auto-consistência interna, não verificação contra a realidade	R14 (rastreabilidade [N] ↔ seção 7) e scripts/validar-codigo.py existem; porém nenhum script da fábrica faz fetch/URL/DOI check (grep urlopen requests doi.org em scripts/*.py = 0 resultados)
Reais	Não	A seção 7 pode conter referência inventada (alucinação) que passa R4/R14/R15. Nada confirma que a fonte existe nem que a afirmação do texto corresponde ao conteúdo citado. validar-codigo.py é análise estática por design (“SEM executar o código do livro”)
Aplicáveis	Parcial	Playbook exige comando executável + entrega com caminho (R-PBK-2/R-PBK-3); livro exige bloco de código (R12). Mas nenhum comando é executado nem testado
Replicáveis	Parcial	Scripts são determinísticos e o playbook é passo a passo; nada prova que o passo reproduz o resultado prometido (o gate do card nunca roda)
Mensuráveis	Não	Nenhum gate exige métrica/indicador com valor e fonte. Um capítulo 100% qualitativo passa na auditoria
Escaláveis	Não	Nenhuma regra sobre limites, condições de contorno ou “até onde funciona” na seção Aplica
Cientificamente embasados	Fraco	Piso de referências (R4) + citação (R10) existem, mas: referências podem ser falsas; não há hierarquia de fontes (primária/secundária/terciária); a citação não é amarrada a afirmações factuais específicas (R10 exige ≥3

Referências de código consultadas

- `scripts/auditar-obra.py` — requisitos R1-R15, sobreposição (Jaccard ≥ 0.45), terminologia; delega para gates dos tipos de extração
- `scripts/validar-codigo.py` — análise estática de sintaxe (`ast.parse`, `node -check`, `bash -n`); sem execução
- `scripts/validar-artefatos.py` — verificação de abertura dos entregáveis (assinatura %PDF, zip, HTML sem placeholder, MAX_PATH)
- `scripts/validar-capa-nivel.py` — badge de nível obrigatório (gate inegociável, já funcional)
- `scripts/validar-playbook.py` (R-PBK-0.8), `validar-lead-magnet.py` (R-LM-1..8), `validar-deck.py` (R-DK-1..5), `validar-emails.py` (R-EM-1..4)
- `scripts/validar-abnt-tcc.py` — pré-textuais NBR 14724/6024/6028
- `scripts/parametros_obra.py` — tamanhos P/M/G/GG/XG, mínimos V3

2. O que implementar — regras sine qua non determinísticas

O padrão a replicar é o que já funciona na casa: gate scriptável + habilitação na skill (ex.: regra 12 do AGENTS.md, `grep 'Autor Digital|centenas de pessoas' vazio; validar-capa-nivel.py` bloqueando a compilação). Aplicar o mesmo molde aos 7 critérios:

1. **validar-referencias.py** — *reais*: extrai URLs/DOIs da seção 7 de cada capítulo, faz HEAD check (cache local + fallback offline “não verificado”), reprova 404/erro de rede. Referências bibliográficas sem URL (livros) passam como registro válido.
2. **validar-codigo.py --executar** (sandbox) — *aplicáveis + replicáveis*: roda blocos Python/bash/JS em venv descartável ou Docker com timeout; reprova erro de execução (hoje só sintaxe). No playbook, o gate de cada card vira smoke test executável.
3. **validar-afirmacoes.py** — *verificáveis + cientificamente embasados*: sentença com dado numérico/percentual/superlativo (regex) sem citação [N] no mesmo parágrafo → violação. Estende R10 de “ ≥ 3 por capítulo” para “toda afirmação factual tem fonte”.
4. **validar-metricas.py** — *mensuráveis*: cada capítulo exige ≥ 1 métrica com valor + unidade + citação adjacente (campo `metricas_obrigatorias` no `sumario_macro.json`).
5. **validar-escala.py** — *escaláveis*: a seção Aplica deve conter limites/condições de contorno/até onde escalar (termos de escala + contraste com o caso base).
6. **Hierarquia de fontes no dossiê** — *cientificamente embasados*: o pesquisador classifica cada fonte (A = peer-reviewed/primária, B = documentação oficial, C = blog/superficial); gate de $\geq 70\%$ das referências por capítulo em A/B, C limitada. Combinado com o item 3, fecha o critério.
7. **Conferência por amostra no revisor-tecnico** — *verificáveis*: reabrir 1 fonte por capítulo via WebFetch e conferir a afirmação (semi-automático, vira checklist obrigatório na skill).
8. **Registro e encadeamento** — novos validadores entram em `scripts/tipos_obra.py` (1 entrada por gate) e `auditar-obra.py --estrito` os encadeia, como já faz com os tipos de extração.

3. Plano de ação — 3 fases

Fase	Entrega	Critérios cobertos
F1 — gates baratos, 100% determinísticos (zero LLM)	validar-referencias.py (link check), validar-codigo.py --executar em sandbox, validar-metricas.py, validar-escala.py + testes com exemplo real de violação	reais, aplicáveis, replicáveis, mensuráveis, escaláveis
F2 — gates estruturais/editoriais	hierarquia A/B/C no dossiê (skill pesquisador), validar-afirmacoes.py, amostragem de conferência no revisor-tecnico	verificáveis, cientificamente embasados
F3 — integração e trava	registro em scripts/tipos_obra.py, encadeamento em auditar-obra --estrito, atualização das skills (estrategista, redator-eita, revisor-tecnico), documentação + relatório de sessão	todos, como regra sine qua non

Regra de ouro: cada gate deve **reprovar com exit 1** (modo --estrito), ter exemplo real de violação nos testes e ficar documentado no RTK scratchpad — como os gates anteriores (capa, personalização por nicho, checkout).

4. Critérios de aceite da entrega

- `python scripts/auditar-obra.py <slug> --estrito` reprova obra cuja referência não existe, cujo código não executa, sem métrica, sem limites de escala ou com afirmação factual sem citação.
- Nenhum material novo (qualquer tipo) passa pela Fase 4 sem os 7 critérios verificados.
- Gates registrados em `scripts/tipos_obra.py` e documentados no AGENTS.md e no RTK scratchpad.